

std::basic_fixed_string

Document #: P3094R1
Date: 2024-03-20
Project: Programming Language C++
Audience: SG16 Unicode
SG18 Library Evolution Working Group Incubator (LEWGI)
Library Evolution Working Group
Reply-to: Mateusz Pusz ([Epam Systems](#))
<mateusz.pusz@gmail.com>

Contents

- 1 Revision history
 - 1.1 Changes since [P3094R0]
- 2 Introduction
- 3 `fixed_string` is an established practice
- 4 Minimal interface requirements
- 5 `fixed_string` design alternatives
 - 5.1 Full `string_view`-like interface
 - 5.2 Mutation interface
 - 5.3 `inplace_string`
 - 5.4 `std::string` with a static storage allocator
 - 5.5 Using `std::array` for storage
 - 5.6 Just wait for the C++ language to solve it
- 6 Design rationale
 - 6.1 Implicit constructor from the array of `CharT`
 - 6.2 Constructor taking the list of characters
- 7 Open questions
 - 7.1 Should the constructor from a string literal be `constexpr`?
 - 7.2 `.view()`
 - 7.3 Should we provide `.at()`?
 - 7.4 Should we provide `std::hash` specializations?
- 8 Implementation experience
- 9 Synopsis
- 10 Acknowledgements
- 11 References

§ 1 Revision history

§ 1.1 Changes since [P3094R0]

- Using `std::array` for storage chapter added.
- Design rationale chapter added.
- Open questions chapter added.
- Synopsis chapter updated:
 - `.view()` replaced with a conversion operator to `std::string_view`,
 - constructor taking `std::integral_constant` replaced with the one that takes iterator and sentinel,
 - `operator==` specification improved,
 - missing alias templates for `wchar_t`, `char8_t`, `char16_t`, and `char32_t` added.

§ 2 Introduction

This paper proposes standardizing a string type that could be used at compile-time as a non-type template parameter (NTTP). This means that such string type has to satisfy the structural type requirements of the C++ language. One of such requirements is to expose all the data members publicly. So far, none of the existing string types in the C++ standard library satisfies such requirements.

This type is intended to serve as an essential building block and be exposed as a part of the public interface of quantities and units library proposed in [P2980R1]. All dimensions, units, prefixes, and constants definitions will use it to provide their textual representation.

§ 3 `fixed_string` is an established practice

`fixed_string` is not just to enable quantities and units library. There are plenty of home-grown `fixed_string` types in the wild already. It is heavily used in low-latency finance, text formatting, compile-time regular expressions, and many other domains.

Today, every project that wants to pass text as NTTP already provides its own version of such type. There are also some that predate C++20 and do not satisfy structural type requirements.

A quick search in GitHub returns plenty of results. Let's mention a few of those:

- [mpusz/mp-units](#)
- [fmtlib/fmt](#)
- [hanickadot/compile-time-regular-expressions](#)
- [tomazos/fixed_string](#) - a reference implementation of [P0259R0] (not a structural type as it is 8 years old)
- [unterumarmung/fixed_string](#)
- [mu001999/fixed_string](#)
- [calebxyz/Fixed_Strings](#)
- [astral-shining/FixedString](#)
- [ynsn/fixed_string](#)

Having such a type in the C++ standard library will prevent reinventing the wheel by the community and reimplementing it in every project that handles text at compile-time.

§ 4 Minimal interface requirements

Such type should at least:

- satisfy structural type requirements,
- be equality comparable and potentially totally ordered,
- store and provide concatenation support for zero-ended strings,
- provide storage that, if set at compile time, would also be available for read-only access at runtime,
- provide at least read-only access to the contained storage.

It does not need to expose a string-like interface. In case its interface is immutable, as it is proposed by the author, we can easily wrap it with `std::string_view` to get such an interface for free.

§ 5 `fixed_string` design alternatives

§ 5.1 Full `string_view`-like interface

We could add a whole `string_view`-like interface to this class, but the author decided not to do it. This will add plenty of overloads that will probably not be used too often by the users of this particular type anyway. Doing that will also add a maintenance burden to keep it consistent with all other string-like types in the library.

If the user needs to obtain a `string_view`-like interface to work with this type, then `std::string_view` itself can easily be used to achieve that:

```
std::basic_fixed_string txt = "abc";
auto pos = std::string_view(txt).find_first_of('b');
```

§ 5.2 Mutation interface

As stated above, this type is read-only. As we can't resize the string, it could be considered controversial to allow mutation of the contents. The only thing we could provide would be support for overwriting specific characters in the internal storage. This, however, is not a common use case for such a type.

If passed as an NTTP, such type would be `const` at runtime, which would disable all the mutation interface anyway.

On the other hand, such an interface would allow running every `constexpr` algorithm from the C++ standard library on such a range at compile time.

Suppose we decide to add such an interface. In that case, it is worth pointing out that wrapping the type in the `std::string_view` would not be enough to obtain a proper string-like mutating interface. As we do not have another string-like reference type that provides mutating capabilities we can end up with the need to implement the entire string interface in this class as well.

This is why the author does not propose adding it to the first iteration. We can always easily add such an interface later if a need arises.

§ 5.3 `inplace_string`

We could also consider providing a full-blown fixed-capacity string class similar to the `inplace_vector` [P0843R9]. It would provide not only full read-write capability but also be really beneficial for embedded and low-latency domains.

Despite being welcomed and valuable in the C++ community, the author believes that such a type should not satisfy the current requirements for a structural type, which is a hard requirement of this use case. If `inplace_string` was used instead, we would end up with separate template instantiations for objects with the same value but a different capacity. We prefer to prevent such a behavior.

§ 5.4 `std::string` with a static storage allocator

We could also try to use `std::string` with a static storage allocator, but this solution does not meet structural type requirements and could be an overkill for this use case, resulting with significantly decreased compile times.

Also, it would be hard or impossible to make the storage created at compile-time to be accessible at runtime in such cases.

§ 5.5 Using `std::array` for storage

We could consider using `std::array` instead, as it satisfies the current requirements for structural types. However, it does not properly construct from string literals and does not provide string-like concatenation interfaces.

§ 5.6 Just wait for the C++ language to solve it

[P2484R0] proposed extending support for class types as non-type template parameters in the C++ language. However, this proposal's primary author is no longer active in C++ standardization, and there have been no updates to the paper in the last two years.

We can't wait for the C++ language to change forever. For example, the quantities and units library will be impossible to standardize without such a feature. This is why the author recommends progressing with the `basic_fixed_string` approach.

§ 6 Design rationale

§ 6.1 Implicit constructor from the array of `CharT`

This constructor is the workhorse of `fixed_string` type. Users will use it to pass string literals to NTTPs. Making the constructors explicit would make the primary use case much more verbose:

Explicit	Implicit
<pre>inline constexpr struct second : named_unit<std::basic_fixed_string{"s"}> {} second; inline constexpr struct metre : named_unit<std::fixed_string{"m"}> {} metre;</pre>	<pre>inline constexpr struct second : named_unit<"s"> {} second; inline constexpr struct metre : named_unit<"m"> {} metre;</pre>

Typically, we agree on making the constructor implicit if:

1. The constructor argument and the type itself represent “the same” abstraction.
2. There is no significant overhead of calling such a constructor.

Point 1. is satisfied as both a string literal and `basic_fixed_string` represent an array of characters ended with `\0`.

Calling such a constructor does not impose any runtime overhead as it is meant to be called at compile time while processing an NTTP argument. This should satisfy point 2. above (see [Should the constructor from a string literal be `constexpr`](#)? for more discussion on this subject).

§ 6.2 Constructor taking the list of characters

This constructor enables a few use cases that are hard to implement otherwise:

1. `std::basic_fixed_string(static_cast<char>('0' + Value))` (used in `mp-units`)
2. `std::fixed_string{'V', 'P', 'B', (char)version}` (credit to Hana Dusíková)

3. std::fixed_string{'W', 'e', 'i', 'r', 'd', '\0', 'b', 'u', 't', ' ', 't', 'r', 'u', 'e'} (credit to Tom Honermann)

§ 7 Open questions

§ 7.1 Should the constructor from a string literal be constexpr?

As stated above, this constructor should be called with a string literal (and not some artificially created array) at compile-time. As it copies characters to the internal buffer, it may impose some overhead when called at runtime. This is why we could consider making it constexpr.

However, in such a case, we should probably add another similar constructor to enable similar initialization at runtime. Such a constructor would be explicit and could take const CharT* and size_type as arguments.

§ 7.2 .view()

Hana Dusíková provided feedback to include .view() member function:

```
template<typename CharT, std::size_t N>
struct basic_fixed_string {
    // ...
    [[nodiscard]] constexpr std::basic_string_view<CharT> view() const noexcept;
};
```

This would be novel in the C++ standard library, but it would simplify the usage of this type:

Before	After
<pre>static_assert(std::string_view(basic_fixed_string("abcd")).find('c') == 2);</pre>	<pre>static_assert(basic_fixed_string("abcd").view().find('c') == 2);</pre>

§ 7.3 Should we provide .at()?

The primary usage of this type is compile-time evaluation, where exceptions can't be thrown and handled. Also, adding .at() makes this type non/partially-freestanding.

§ 7.4 Should we provide std::hash specializations?

It is probably not the best idea to store values of this type in associative containers, as the length of the contained text is encoded in the type identifier. This means that such a container would only be able to store strings of exactly the same size.

§ 8 Implementation experience

This particular interface is implemented and successfully used in the [mp-units](#) project.

§ 9 Synopsis

fixed_string is a read-only text container. It satisfies structural type requirements, its size is fixed as one of the class template parameters, and it has a minimalistic range/view-like interface:

```
template<typename CharT, size_t N>
struct basic_fixed_string {
    CharT data_[N + 1] = {}; // exposition only

    using value_type = CharT;
    using pointer = CharT*;
    using const_pointer = const CharT*;
    using reference = CharT&
    using const_reference = const CharT&
    using const_iterator = const CharT*;
    using iterator = const_iterator;
    using size_type = size_t;
    using difference_type = ptrdiff_t;

    constexpr explicit(false) basic_fixed_string(const CharT (&txt)[N + 1]) noexcept;

    template<std::input_iterator It, std::sentinel_for<It> S>
        requires std::convertible_to<std::iter_value_t<It>, CharT>
    constexpr explicit basic_fixed_string(It first, S last) noexcept;

    template<convertible_to<CharT>... Rest>
        requires(1 + sizeof...(Rest) == N)
    constexpr explicit basic_fixed_string(CharT first, Rest... rest) noexcept;

    [[nodiscard]] constexpr bool empty() const noexcept;
```

```

[[nodiscard]] constexpr size_type size() const noexcept;
[[nodiscard]] constexpr const_pointer data() const noexcept;
[[nodiscard]] constexpr const CharT* c_str() const noexcept;
[[nodiscard]] constexpr value_type operator[](size_type index) const noexcept;

[[nodiscard]] constexpr const_iterator begin() const noexcept;
[[nodiscard]] constexpr const_iterator cbegin() const noexcept;
[[nodiscard]] constexpr const_iterator end() const noexcept;
[[nodiscard]] constexpr const_iterator cend() const noexcept;

[[nodiscard]] constexpr operator basic_string_view<CharT>() const noexcept;

template<size_t N2>
[[nodiscard]] constexpr friend basic_fixed_string<CharT, N + N2> operator+(const basic_fixed_string& lhs,
                                                                    const basic_fixed_string<CharT, N2>& rhs)
    noexcept;

[[nodiscard]] constexpr bool operator==(const basic_fixed_string&) const = default;
template<size_t N2>
[[nodiscard]] friend constexpr bool operator==(const basic_fixed_string&, const basic_fixed_string<CharT, N2>&) {
    return false; }

template<size_t N2>
[[nodiscard]] friend constexpr auto operator<=>(const basic_fixed_string& lhs, const basic_fixed_string<CharT, N2>&
    rhs);

template<typename Traits>
friend basic_ostream<CharT, Traits>& operator<<(basic_ostream<CharT, Traits>& os, const basic_fixed_string<CharT, N>&
    str);
};

template<typename CharT, size_t N>
basic_fixed_string(const CharT (&str)[N]) -> basic_fixed_string<CharT, N - 1>;

template<typename CharT, convertible_to<CharT>... Rest>
basic_fixed_string(CharT, Rest...) -> basic_fixed_string<CharT, 1 + sizeof...(Rest)>;

template<size_t N>
using fixed_string = basic_fixed_string<char, N>;

template<size_t N>
using fixed_wstring = basic_fixed_string<wchar_t, N>;

template<size_t N>
using fixed_u8string = basic_fixed_string<char8_t, N>;

template<size_t N>
using fixed_u16string = basic_fixed_string<char16_t, N>;

template<size_t N>
using fixed_u32string = basic_fixed_string<char32_t, N>;

template<typename CharT, size_t N>
struct formatter<basic_fixed_string<CharT, N>> : formatter<basic_string_view<CharT>> {};

```

§ 10 Acknowledgements

Special thanks and recognition goes to [Epam Systems](#) for supporting Mateusz’s membership in the ISO C++ Committee and the production of this proposal.

The author would also like to thank Hana Dusíková and Nevin Liber for providing valuable feedback that helped him shape the final version of this document.

§ 11 References

- [P0259R0] Michael Price, Andrew Tomazos. 2016-02-12. `fixed_string`: a compile-time string.
<https://wg21.link/p0259r0>
- [P0843R9] Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel. 2023-09-14. `inplace_vector`.
<https://wg21.link/p0843r9>
- [P2484R0] Richard Smith. 2021-11-17. Extending class types as non-type template parameters.
<https://wg21.link/p2484r0>
- [P2980R1] Mateusz Pusz, Dominik Berner, Johel Ernesto Guerrero Peña, Charles Hogg, Nicolas Holthaus, Roth Michaels, Vincent Reverdy. 2023-11-28. A motivation, scope, and plan for a quantities and units library.
<https://wg21.link/p2980r1>
- [P3094R0] Mateusz Pusz. 2024-02-05. `std::basic_fixed_string`.
<https://wg21.link/p3094r0>